SYSTEM ARCHITECTURE SPECIFICATION

SLOTBOOK

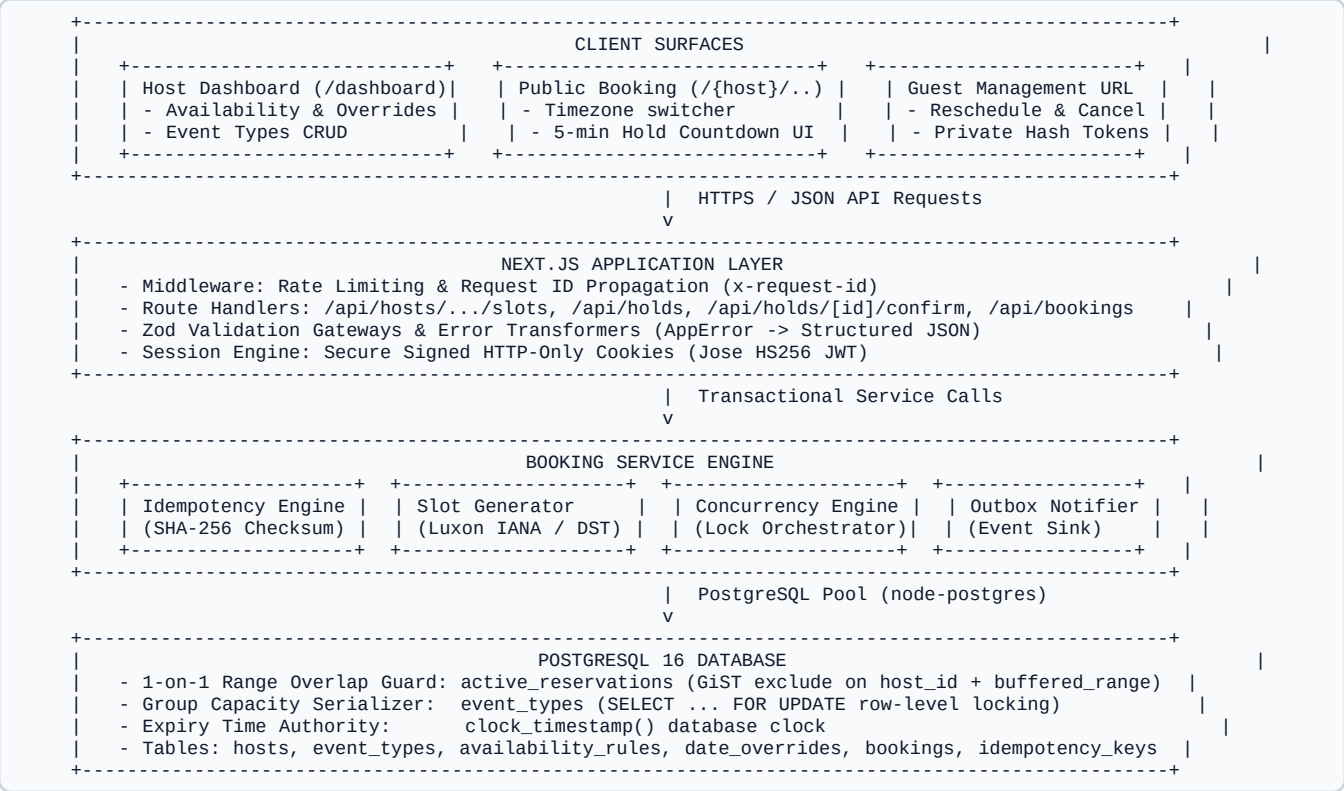
Reliable Appointment Booking Platform

Zero Double-Booking Guarantee via PostgreSQL Engine Invariants

Technology Stack:	TypeScript, Next.js App Router, PostgreSQL 16, Drizzle ORM
Concurrency Enforcement:	PostgreSQL GiST Exclusion Constraints & Row-Level Locks
Deployment:	Docker Compose Multi-Container Orchestration
Repository:	https://github.com/tiruk007/slotbooking.git
Branch & Target:	main (commit c56a6bc)
Document Version:	1.0.0 (Production Architecture Review)
Date of Preparation:	November 2026

1. System Architecture

Slotbook is structured into three client surfaces interacting with a Next.js App Router backend and an authoritative PostgreSQL 16 database. Concurrency correctness is delegated directly to the database storage engine rather than fragile application-tier in-memory locks or external distributed state caches.

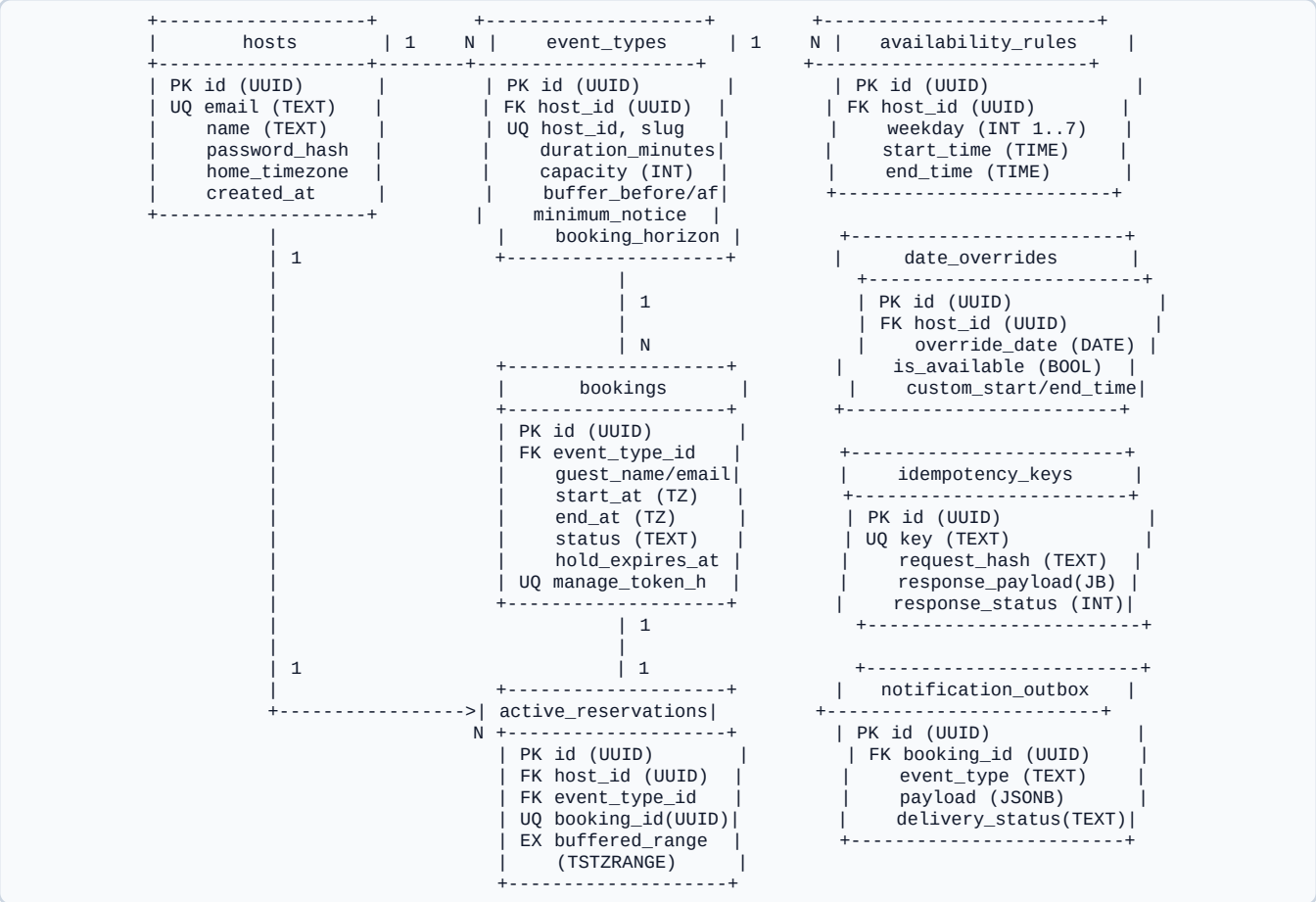


Request Flow & Component Responsibilities

1. **Guest Booking Flow**: The unauthenticated guest opens `/{host}/{event-slug}`. The Next.js frontend queries `/api/hosts/{host}/events/{slug}/slots?tz={guestTz}`. The **Slot Generator** evaluates the host's weekly rules, date overrides, existing confirmed/held bookings, and buffers in the host's local timezone using Luxon, returning available slots in UTC ISO-8601.
2. **Hold Placement**: When a slot is clicked, the guest browser generates an `Idempotency-Key` and issues `POST /api/holds`. The **Booking Service** initiates a PostgreSQL transaction, acquires an exclusive lock, purges stale holds, validates capacity, inserts the hold, and inserts an active reservation claim.
3. **Hold Confirmation**: The guest supplies their name and email, calling `POST /api/holds/{id}/confirm`. The database transaction verifies `clock_timestamp() <= hold_expires_at`, generates a 64-character private management token, stores its SHA-256 hash in `manage_token_hash`, and writes a `BOOKING_CONFIRMED` event to the `notification_outbox`.
4. **Host Dashboard & Administration**: Authenticated hosts configure weekly availability windows (multiple allowed per weekday) and date-specific overrides. Session tokens are verified via signed HTTP-only cookies; mutation operations enforce host ownership at the database query level.

2. Database Architecture & Schema Specification

The database model is normalized into 8 core tables with strict foreign keys, cascade rules, unique indexes, and PostgreSQL-specific range exclusion constraints.

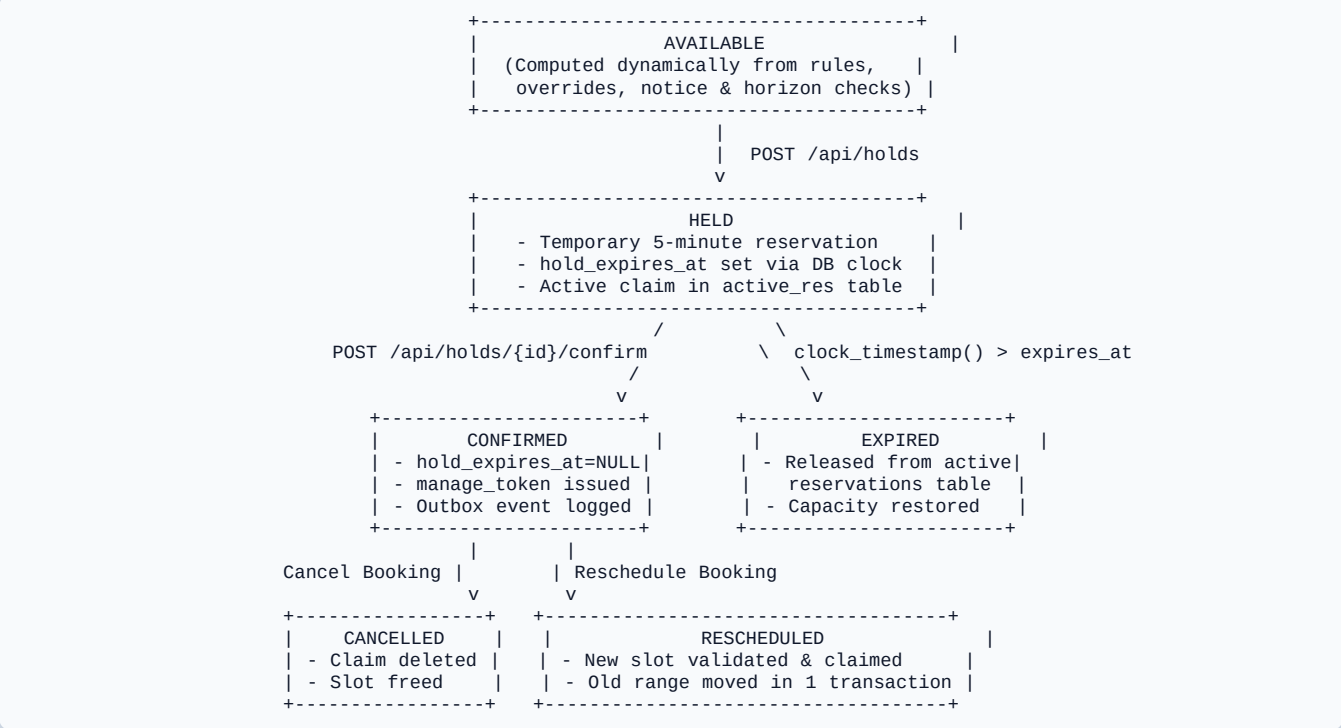


Entity Definitions & Relational Integrity

Table	Primary Key	Constraints, Indexes & Characteristics
`hosts`	`id (UUID)`	`email` UNIQUE; `home_timezone` defaults to 'UTC'; B-Tree index on email.
`event_types`	`id (UUID)`	`UNIQUE(host_id, slug)`; duration 15..120; capacity >= 1; buffers >= 0.
`availability_rules`	`id (UUID)`	`CHECK(start_time < end_time)`; weekday 1..7 (Luxon ISO standard).
`date_overrides`	`id (UUID)`	`override_date` DATE; `is_available` boolean; custom time bounds check.
`bookings`	`id (UUID)`	`TIMESTAMPTZ` start/end bounds; status enum check; SHA-256 token hash index.
`active_reservations`	`id (UUID)`	**GiST exclusion constraint** (`host_id = WITH, buffered_range && WITH`).
`idempotency_keys`	`id (UUID)`	`key` UNIQUE; stores request SHA-256 hash and response payload JSONB.
`notification_outbox`	`id (UUID)`	Transactional outbox; status enum ('PENDING', 'DELIVERED', 'FAILED').

3. Booking Lifecycle & State Transitions

Booking states follow an immutable state machine designed to prevent stale holds from artificially consuming capacity while ensuring clients can retry and manage appointments securely.



Lifecycle State Transition Semantics

1. **AVAILABLE → HELD**: Triggered by `POST /api/holds` with `eventTypeId` and `startAt`. The server sets `status = 'HELD'` and `hold_expires_at = clock_timestamp() + interval '5 minutes'`. If `capacity = 1`, an active claim is registered in `active_reservations` with `buffered_range = tstzrange(start - buffer_before, end + buffer_after, '[)')`.
2. **HELD → CONFIRMED**: Triggered by `POST /api/holds/{id}/confirm` with guest name and email. The transaction locks the booking row (`FOR UPDATE`) and validates that `clock_timestamp() <= hold_expires_at`. Generates a cryptographically secure 64-character hex token (`crypto.randomBytes(32)`), stores its SHA-256 hash in `manage_token_hash`, clears `hold_expires_at`, and emits `BOOKING_CONFIRMED`.
3. **HELD → EXPIRED**: Occurs automatically when `clock_timestamp() > hold_expires_at`. Any subsequent slot computation or hold creation purges expired records from `active_reservations` in its pre-flight step.
4. **CONFIRMED → RESCHEDULED**: Moves the appointment in a single atomic transaction. The new slot is checked and claimed; if the target slot collides, the entire transaction rolls back and the original booking remains untouched.
5. **CONFIRMED / HELD → CANCELLED**: Deletes the claim from `active_reservations`, sets `status = 'CANCELLED'`, records `cancellation_reason`, and emits `BOOKING_CANCELLED`.

4. Double-Booking Prevention & Concurrency Safety

The primary architectural challenge in scheduling platforms is the **check-then-insert race condition**: two concurrent requests inspect availability, both determine that a slot is open, and both insert reservations, exceeding capacity. Slotbook completely eliminates this vulnerability using database engine primitives.

One-on-One Bookings (Capacity = 1): GiST Range Exclusion

For single-capacity bookings, Slotbook leverages PostgreSQL's `btree_gist` extension to establish a generalized exclusion constraint on the `active_reservations` table:

```
CREATE EXTENSION IF NOT EXISTS "btree_gist";

CREATE TABLE active_reservations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  host_id UUID NOT NULL REFERENCES hosts(id) ON DELETE CASCADE,
  event_type_id UUID NOT NULL REFERENCES event_types(id) ON DELETE CASCADE,
  booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE UNIQUE,
  buffered_range TSTZRANGE NOT NULL,
  CONSTRAINT exclude_host_schedule_overlap EXCLUDE USING gist (
    host_id WITH =,
    buffered_range WITH &&
  )
);
```

Why this is mathematically uncheatable:

- When transaction A and transaction B attempt to reserve overlapping slots, PostgreSQL's GiST index detects the predicate collision (`buffered_range WITH &&`) for the same `host_id`.
- The second transaction is aborted with PostgreSQL error code **`23P01` (`exclusion_violation`)**.
- Buffers are embedded inside the `tstzrange`: `[start_at - buffer_before, end_at + buffer_after)`. Thus, buffer conflicts are prevented at the index level across different event types for the same host.

Group Bookings (Capacity > 1): Serialized Row-Level Locking

For events accommodating multiple guests (e.g., workshops with capacity = 4), range exclusion cannot be used directly because multiple reservations are permitted in the same interval. Slotbook implements serialized row-level locking:

```
-- Step 1: Serialize competing transactions for this event type
SELECT id, host_id, capacity FROM event_types WHERE id = $1 FOR UPDATE;

-- Step 2: Purge stale holds using database time
WITH expired_holds AS (
  UPDATE bookings SET status = 'EXPIRED', updated_at = clock_timestamp()
  WHERE status = 'HELD' AND hold_expires_at <= clock_timestamp() RETURNING id
)
DELETE FROM active_reservations WHERE booking_id IN (SELECT id FROM expired_holds);

-- Step 3: Count active reservations
SELECT count(*) FROM bookings
WHERE event_type_id = $1 AND status IN ('HELD', 'CONFIRMED')
  AND start_at = $2::timestamptz
  AND (hold_expires_at IS NULL OR hold_expires_at > clock_timestamp());

-- Step 4: Insert only if count < capacity, then COMMIT;
```

Under PostgreSQL's `READ COMMITTED` isolation level, the `SELECT ... FOR UPDATE` causes concurrent transactions to queue. When the lock releases, the queued transaction re-reads the updated row, re-executes the count query, observes the committed booking, and halts if capacity is exhausted.

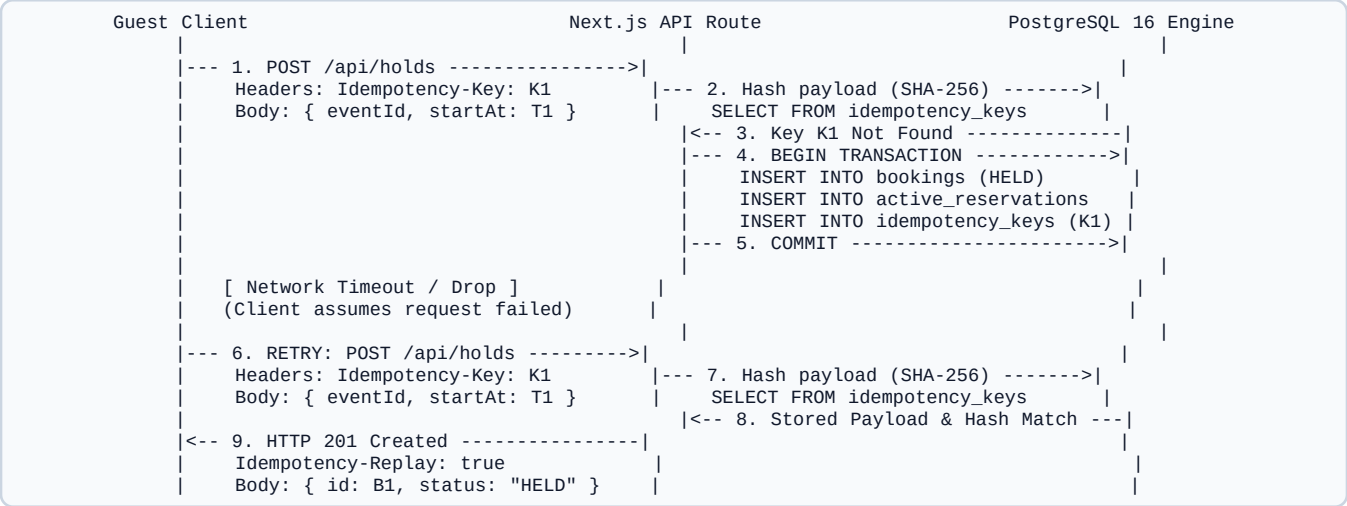
5. Idempotency & Network Failure Handling

Network timeouts and retry storms can lead to duplicate reservations. Slotbook implements an `Idempotency-Key` header contract on `POST /api/holds` and safe confirmation retries.

The Idempotency Contract

Condition	Behavior & Response Code
New Request	Executes transaction, stores request checksum and response JSONB; returns HTTP 201 Created.
Identical Retry	Matches stored SHA-256 payload hash; replays stored response with `Idempotency-Replay: true` (HTTP 201).
Payload Conflict	Identical key sent with different parameters; rejects immediately with HTTP 409 Conflict (`IDEMPOTENCY_CONFLICT`).
Concurrent Race	Two concurrent requests with the identical key hit the unique constraint on `idempotency_keys.key` (`23505`); one succeeds, the other retries or receives 409.

Sequence Diagram: Identical Retry Scenario



Safe Confirmation Retries

If a guest double-clicks or retries `POST /api/holds/{id}/confirm`, the service checks if the booking is already in status `CONFIRMED` with the matching guest email. If true, it returns the confirmation payload idempotently without generating a second management token or failing.

6. Time-Zone Architecture & Daylight Saving Time (DST)

Slotbook implements full IANA time-zone handling via Luxon. The architecture decouples the host's operating calendar from the guest's presentation timezone and database storage.

Core Invariants

1. **Host IANA Identifier**: Stored as an IANA timezone string (e.g., `Asia/Kolkata`, `America/New_York`).
2. **Local Calendar Evaluation**: Operating windows (`09:00:00–17:00:00`) and date overrides are evaluated in the host's local wall clock.
3. **Absolute Database Storage**: All booking instants are converted to UTC and stored in PostgreSQL as `TIMESTAMPTZ`.
4. **Zero Fixed-Offset Arithmetic**: Offsets are never calculated by adding constant integer hours (e.g., `+5.5` or `-5`). All calculations use Luxon's IANA zone engine.

End-to-End Cross-Timezone Example

Host Local Time	PostgreSQL Storage	Guest Browser Time
Zone: `Asia/Kolkata` (IST)	Type: `TIMESTAMPTZ` (UTC)	Zone: `America/New_York` (EST)
Friday, 09:30 AM IST Evaluated against Mon–Fri 09:00–12:00 availability rule	**`2026-11-06T04:00:00.000Z`** Universal instant stored without ambiguity	**Thursday, 11:00 PM EST** Rendered seamlessly in guest's calendar

Daylight Saving Time (DST) Policies

DST Transition Event	Slotbook Resolution Policy & Implementation
Spring-Forward Gap (Nonexistent local wall clock)	In `America/New_York`, clocks skip from 02:00 to 03:00 on the second Sunday in March. Times such as 02:30 do not exist. **Policy**: `isValidWallTime()` tests roundtrip fidelity. If Luxon shifts the requested hour/minute to accommodate the gap, the time is rejected and excluded from generated slots.
Fall-Back Overlap (Ambiguous local wall clock)	On the first Sunday in November, clocks fall back from 02:00 EDT to 01:00 EST. The hour 01:00–01:59 occurs twice. **Policy**: Resolved consistently according to standard IANA rules to Standard Time (UTC-5, offset −300 minutes). All API responses emit ISO-8601 UTC strings with `Z`, preventing downstream ambiguity.

7. REST API Architecture & Endpoints

All endpoints emit and accept JSON, enforce runtime Zod validation, return standard HTTP status codes, and output structured error objects: `{ "error": { "code": "...", "message": "..." } }`.

Method	Endpoint	Purpose & Validation
GET	/api/hosts/{host}/events/{slug}/slots	Generates available slots for next 30 days in guest timezone `?tz=...`
POST	/api/holds	Places 5-minute hold on slot; supports `Idempotency-Key` header.
POST	/api/holds/{id}/confirm	Confirms held slot with guest name & email; issues management token.
POST	/api/bookings/{token}/reschedule	Atomically moves confirmed booking to new `newStartAt` ISO timestamp.
POST	/api/bookings/{token}/cancel	Cancels appointment with optional reason; frees active reservation claim.
GET	/api/health	System health check probing database connection and `clock_timestamp()`.

Endpoint Specification & Payloads

Create Temporary Hold

```
POST /api/holds HTTP/1.1
Content-Type: application/json
Idempotency-Key: 7b9d5c41-8c82-4f51-b847-1930129bc481

{
  "eventTypeId": "49d1390c-5a88-4d63-bf28...",
  "startAt": "2026-11-10T04:00:00.000Z"
}
```

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "c1f7b029-4501-4be6-a97e-128919db450a",
  "status": "HELD",
  "holdExpiresAt": "2026-11-10T04:05:00.000Z"
}
```

Confirm Hold & Receive Token

```
POST /api/holds/c1f7b029.../confirm HTTP/1.1
Content-Type: application/json

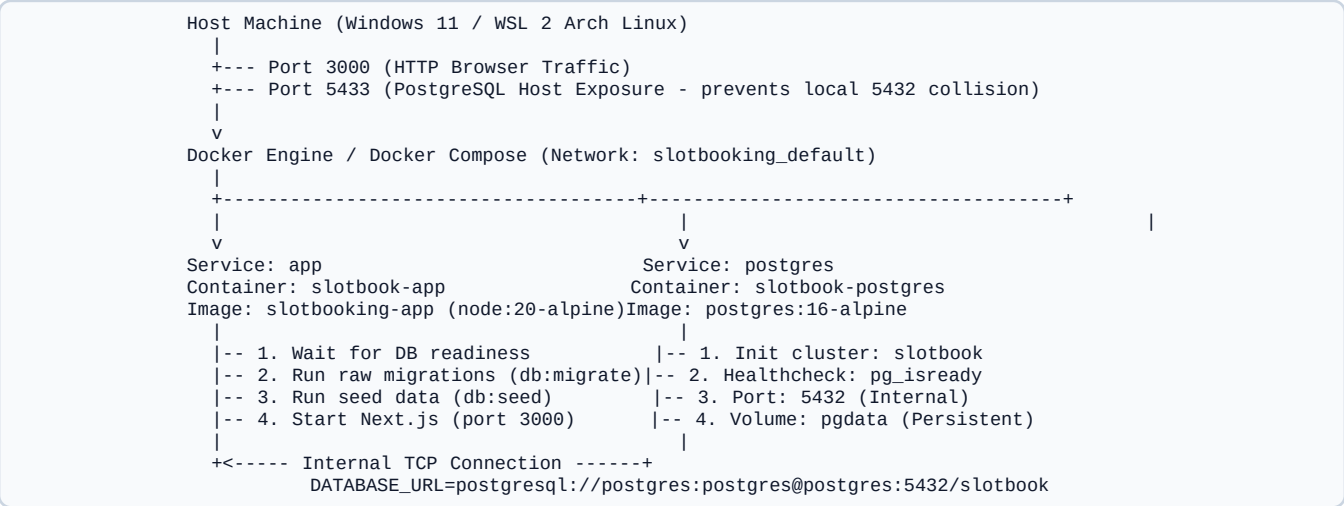
{
  "guestName": "Sarah Connor",
  "guestEmail": "sarah@example.com"
}
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": "c1f7b029-4501-4be6-a97e-128919db450a",
  "status": "CONFIRMED",
  "manageToken": "9e3c9217e573a4b9c1d6...",
  "manageUrl": "/bookings/9e3c9217e573..."
}
```

8. Docker Deployment Architecture

The application is containerized using a clean, reproducible multi-container topology orchestrating PostgreSQL 16 Alpine and the Next.js production runtime.



Container Specifications & Lifecycle Coordination

Service	Base Image	Responsibilities & Configuration
`postgres`	`postgres:16-alpine`	• Runs PostgreSQL 16 engine with internal port `5432`. • Host port mapped to `${POSTGRES_PORT:-5433}:5432` to avoid host collisions. • Persistent volume `pgdata` mounted at `/var/lib/postgresql/data`. • Health check executes `pg_isready -U postgres -d slotbook` every 3s.
`app`	`node:20-alpine`	• Multi-stage build (`deps → builder → runner`). • Depends on `postgres` with condition `service_healthy`. • `docker-entrypoint.sh` coordinates migration execution, database seeding, and production server boot.

9. Automated Testing & Reliability Evidence

Slotbook's test suite operates directly against a real PostgreSQL instance to verify engine-level invariants. In-memory databases or mocks are deliberately avoided.

Test Suite Execution Results (`npm test`)

```
 RUN  v2.1.9 /home/tirum/Cluster/slotbooking

 ✓ tests/concurrency.test.ts (9 tests) 621ms
   ✓ Test 1 — Concurrent booking: 20 requests for capacity=1 slot -> exactly 1 succeeds (409 on 19)
   ✓ Test 2 — Group capacity: 20 requests for capacity=3 slot -> exactly 3 succeed (409 on 17)
   ✓ Test 3 — Idempotency: duplicate key replays original result; different payload returns 409
   ✓ Test 4 — Expired hold: expired hold frees capacity and allows replacement reservation
   ✓ Test 5 — Confirmation race: booking cannot transition to confirmed after DB expiry
   ✓ Test 6 — Rescheduling: moves reservation atomically; collision leaves original booking intact
   ✓ Test 7 — Time zones: Asia/Kolkata conversion & America/New_York DST transitions
 ✓ tests/api.test.ts (1 test) 63ms
   ✓ handles full lifecycle: hold -> confirm -> reschedule -> cancel with outbox events

 Test Files  2 passed (2)
      Tests  10 passed (10)
   Duration  1.59s (transform 71ms, tests 684ms)
```

Evidence of Double-Booking Protection

Test Scenario	Load	Actual Verified Invariant
Capacity = 1 Concurrency	20 parallel calls	**Exactly 1** reservation created; 19 rejected with 409 (`SLOT_NOT_AVAILABLE`). Verified via database count queries on `bookings` and `active_reservations`.
Group Capacity = 3	20 parallel calls	**Exactly 3** reservations created; 17 rejected with 409. Confirmed row count = 3 in PostgreSQL.
Idempotent Replay	Same key + payload	Returns identical reservation payload with `isIdempotentReplay: true`. Database record count remains 1.
Reschedule Collision	Overlap target slot	Target collision throws 409; original booking remains `CONFIRMED` at original start time.

Known Limitations & Future Enhancements

1. **Asynchronous Outbox Dispatcher**: Events are currently staged in `notification_outbox` in the same transaction as state changes. A decoupled background daemon can consume records with `LISTEN/NOTIFY` to dispatch external emails via SendGrid or SES.
2. **External Calendar Integration**: Bi-directional synchronization with Google Calendar or Microsoft Outlook via Webhooks can be connected into outbox state changes without modifying core reservation invariants.